```
  enumcolors { black, red, green, blue, violet, white } c;
     /* enumerator blue = 3 now hides outer declaration
of int blue */
  structcolors { int i, j; }; // ILLEGAL: colors
duplicate tag
     double red = 2;      // ILLEGAL: redefinition of red
  }
  blue = 37;                    // back in int blue scope
```

Void Type

void is a special type indicating the absence of any value. There are no objects of void; instead, void is used for deriving more complex types.

Void Functions

Use the void keyword as a function return type if the function does not return a value.

```
  void print_temp(char temp) {
  Lcd_Out_Cp("Temperature:");
  Lcd_Out_Cp(temp);
  Lcd_Chr_Cp(223);  // degree character
  Lcd_Chr_Cp('C');
  }
```

Use void as a function heading if the function does not take any parameters. Alternatively, you can just write empty parentheses:

```
  main(void) { // same as main()
    ...
  }
```

4.2.2 Generic Pointers

Pointers can be declared as void, meaning that they can point to any type. These pointers are sometimes called generic.

Derived Types

The derived types are also known as structured types. These types are used as elements in creating more complex user-defined types. The derived types include arrays, pointers, structures, unions, arrays.

Array is the simplest and most commonly used structured type. Variable of array type is actually an array of objects of the same type. These objects